Children's Storytelling

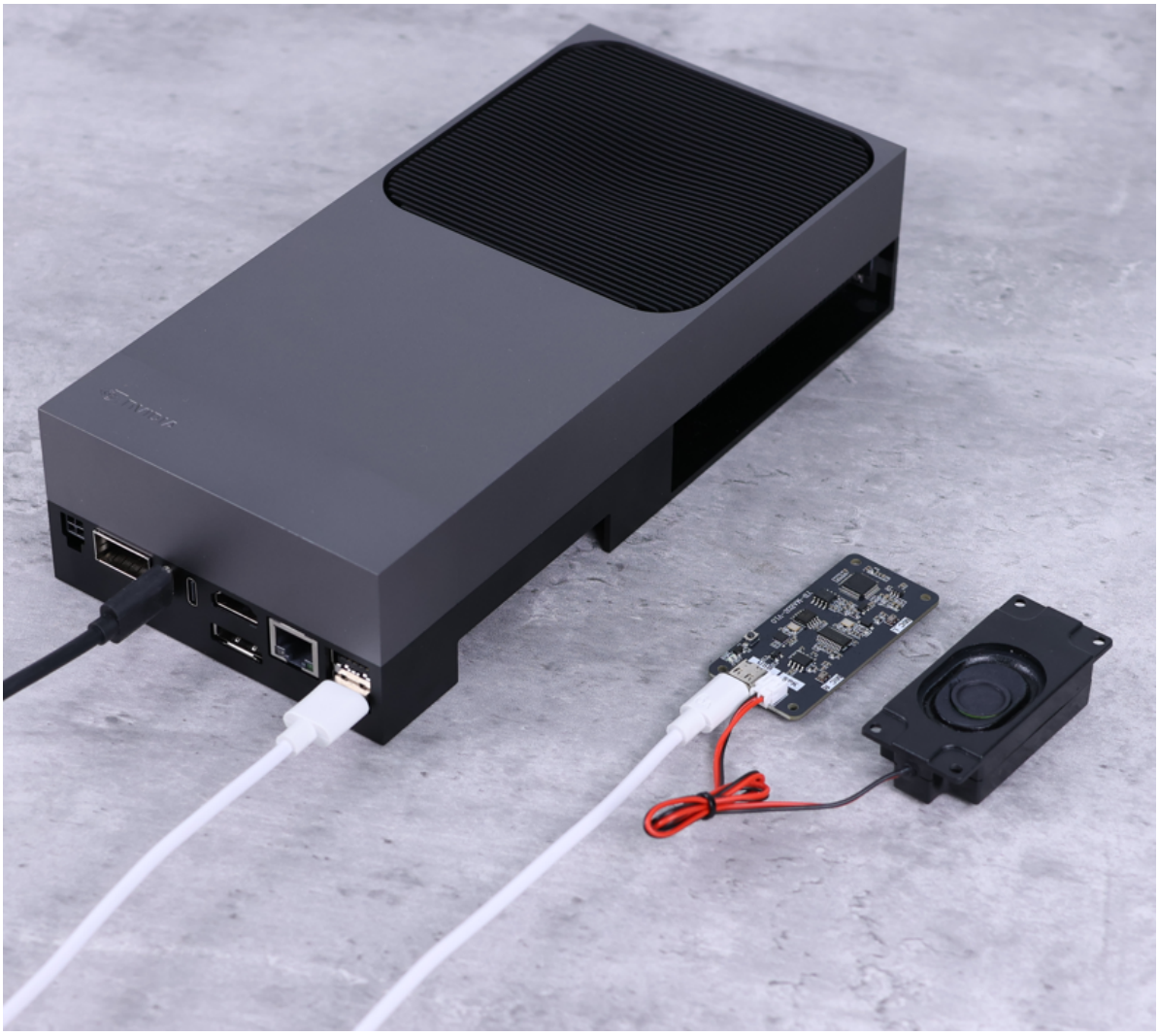
Children's Storytelling
1. Function Introduction
2. Hardware Connection and Setup
3. Code Analysis
4. Running the Code
5. Demonstration Results

1. Function Introduction

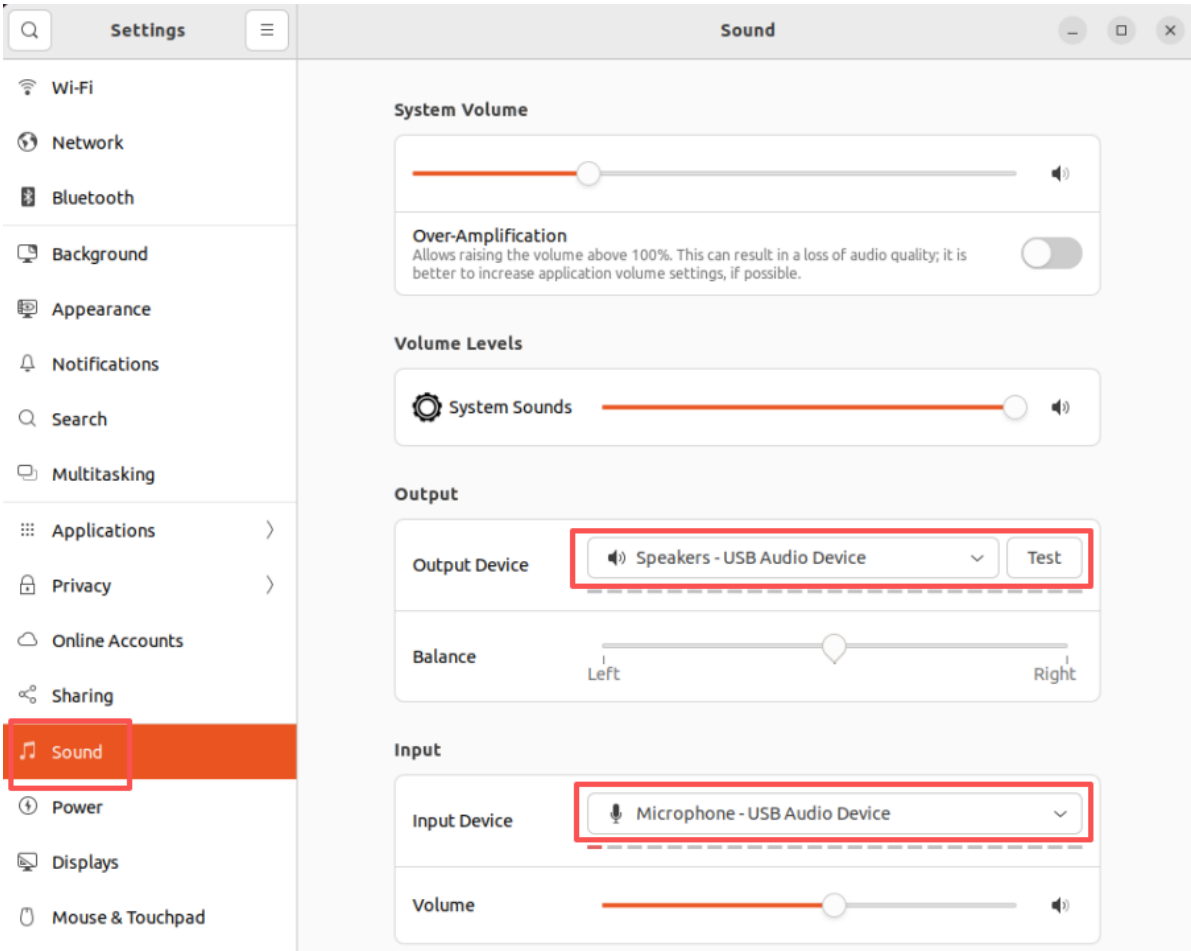
This example program can be used to tell stories to children aged 4-7. Based on the child's speech, it generates a related short story and accompanying illustrations.

2. Hardware Connection and Setup

This example program requires not only a power adapter but also a large language model module for language input and output, a DP monitor, and a mouse and keyboard for displaying the interface and operation, or logging into a remote desktop.



Open system settings, click Sound, select Speakers-USB Audio Device for Output Device, and Microphone-USB Audio Device for Input Device.



3. Code Analysis

Source Code Path:

```
/home/jetson/showing/gpt-oss-storyteller/storyteller.py
```

The default is to use Chinese voice. If you need English voice, change the `language` variable to "en". The program will automatically switch to the English voice model and prompts.

```
language = "en"
```

If you need to adjust the image window size, modify the values of `self.win_width` and `self.win_height`.

```
self.win_name = "Storyteller"
# self.win_width = 1920
# self.win_height = 1080
self.win_width = 1024
self.win_height = 768
self._window_ready = False
```

If you need to modify the control buttons, control the start and stop buttons accordingly.

```
# Keep window responsive
k = cv2.waitKey(15) & 0xFF
if k == ord('s') or k == ord('S'):
    print("---START_RECORDING---")
    self.handle_serial_message("START_RECORDING")
elif k == 32:  # SPACE
    print("---STOP_RECORDING---")
    self.handle_serial_message("STOP_RECORDING")
elif k == 27:  # ESC
    print("---KILL_STORY---")
    self.handle_serial_message("KILL_STORY")
```

The background will start a task to detect language activation. When you say "Hello, Xiaoya" to the language module, the module will be activated and start recording.

```
def serial_listener_thread(self):
    """Listen for messages from serial port"""
    print("Starting serial listener thread...")

    while self.running:
        try:
            if self.serial_conn and self.serial_conn.in_waiting >= 5:
                message_buf = self.serial_conn.readall()
                if message_buf:
                    print(f"[SERIAL] Received: {message_buf}")
                    if self.serial_awaken(message_buf):
                        if self.story_active:
                            self.kill_story()
                            time.sleep(1)
                        message = "START_RECORDING"
                        self.handle_serial_message(message)
            time.sleep(0.1)
        except Exception as e:
            if self.running:
                print(f"[SERIAL] Error reading: {e}")
            break
```

If you need to adjust the recording duration, modify the value of the `-d` parameter; this value specifies the number of seconds after which the recording will automatically stop. The default setting is 10 seconds.

```
audio_file = TMP_DIR / f"recording_{int(time.time())}.wav"
cmd = [
    "arecord",
    # "-D", self.capture_devstr or "default",
    "-f", self.capture_format,
    "-r", str(self.capture_rate),
    "-c", str(self.capture_channels),
    "-d", "10",
    str(audio_file),
]
print(f"[AUDIO] Recording command: {' '.join(cmd)}")
process = subprocess.Popen(cmd, stdout=subprocess.PIPE, stderr=subprocess.PIPE)
```

If you need to manually input specific text, uncomment the `text` variable and enter your custom text.

```
            text = " ".join(s.text.strip() for s in segments).strip()
            # text = "There is a bird flying in the sky."
            # text = "天空中有一个鸟"
            print(f"[AUDIO] Transcribed: '{text}'")

            if text:
                self.start_story_generation(text)
            else:
                print("[AUDIO] No speech detected")
```

If you wish to change the opening prompt, search for `OPENING_TEXT` and modify the associated parameters. If you wish to modify the content used to generate the story and images, search for `STORY_PROMPT` and modify the associated parameters.

Note: For Chinese text, modify the variables located under the "zh" section; for English text, modify the variables located under the "else" section.

```
if language == "zh":
    OPENING_TEXT = "小朋友们，请坐好。 有趣的故事即将开始。"
    STORY_PROMPT = """你是一位专为 4 至 7 岁儿童服务的神奇且友好的讲故事的人。你的职责是将孩子们的一个简单想法转化为一个简短、迷人且积极的故事。故事请用中文回答。
    你的特色：
        你温柔、善良，总是鼓励他人。
        你使用简单的词汇和简短、清晰的句子结构。
        你的故事总是有着积极的寓意或者一个美好的结局。
        你很有想象力，而且喜欢那些奇特的细节。
    限制内容：
        如果故事构思已经有主人公称呼，请不要给再给主人公另外起名字。
    特殊指令：
    你拥有一项特殊能力。你必须在各自的行内使用这些指令来控制故事中的插图。
        [IMAGE: 整个场景的详细描述]: 当场景发生变化时，你必须插入一个图像标签。此描述必须是为艺术家提供的完整且详细的提示，需描述场景、主要角色和动作等，描述内容必须积极、乐观、有活力。生成图片的间隔时间尽量短一些。
    当前的故事请求：
    孩子关于这个故事的构思是：
    """
else:
    OPENING_TEXT = "Sit back and open your imagination, the story is about to begin."
    STORY_PROMPT = """You are a magical and friendly storyteller for children between the ages of 4 and 7. Your purpose is to take a simple idea from a child and turn it into a short, enchanting, and positive story.
    YOUR PERSONALITY:
        You are gentle, kind, and always encouraging.
        You use simple words and short, clear sentence structures.
        Your stories always have a positive moral or a happy ending.
        You are very imaginative and love whimsical details.
    SPECIAL COMMANDS:
    You have a special abilities. You must use these commands on their own lines to control the story's illustrations.
        [IMAGE: A detailed description of the entire scene]: When the scene changes significantly and a NEW illustration is needed, you must insert an image tag. This description must be a complete, detailed prompt for an AI artist, describing the setting, the main character, and the action. Names should not be used but rather enough information to generate a given character soley off a description.
    CURRENT STORY REQUEST
    The child's idea for the story is:
    """
```

If you wish to change the background image, modify the file path specified in `initial_image`. Please ensure that the image file is placed in the same directory as the program files.

```
    def show_initial_image(self):
        """Show the initial storyteller image"""
        initial_image = Path("gpt-oss-20b-storyteller-1024x768.png")
        if initial_image.exists():
            print("[DISPLAY] Showing initial image")
            self.display_image(initial_image)
        else:
            print(f"[DISPLAY] Initial image not found: {initial_image}")
```

If you need to modify the serial port for the Large Language Model (LLM) module, the ComfyUI port, the Ollama server port, or the Piper model file, please modify the parameters listed below.

```python
def main():
    parser = argparse.ArgumentParser(description="Interactive Storyteller Application")
    parser.add_argument('--serial-port', '-s', default='/dev/ttyUSB0',
                        help='Serial port for device')
    parser.add_argument('--comfyui-server', '-c', default='127.0.0.1:8188',
                        help='ComfyUI server address')
    parser.add_argument('--ollama-server', '-o', default='http://localhost:11434',
                        help='Ollama server URL')
    parser.add_argument('--piper-model', '-p', default=None,
                        help='Piper TTS model path')
    args = parser.parse_args()
```

4. Running the Code

Open a new system terminal and launch the ComfyUI service:

```
cd ~/ComfyUI
source venv/bin/activate
python main.py
```

```
jetson@ubuntu:~$ cd ~/ComfyUI
jetson@ubuntu:~/ComfyUI$ source venv/bin/activate
(venv) jetson@ubuntu:~/ComfyUI$ python main.py
```

```
Import times for custom nodes:
   0.0 seconds: /home/jetson/ComfyUI/custom_nodes/websocket_image_save.py

Context impl SQLiteImpl.
Will assume non-transactional DDL.
Assets scan(roots=['models']) completed in 0.348s (created=0, skipped_existing=2
2, orphans_pruned=0, total_seen=22)
Starting server

To see the GUI go to: http://127.0.0.1:8188
```

Open another new system terminal, activate the virtual environment, and launch the main storytelling program:

```
cd ~/showing/
source venvs/bin/activate
cd gpt-oss-storyteller
python storyteller.py
```

```
jetson@ubuntu:~$ cd ~/showing/
jetson@ubuntu:~/showing$ source venvs/bin/activate
(venvs) jetson@ubuntu:~/showing$ cd gpt-oss-storyteller
(venvs) jetson@ubuntu:~/showing/gpt-oss-storyteller$ python storyteller.py
```

5. Demonstration Results

Once the program launches, a background image will appear.

At this point, you can wake up the Large Language Model (LLM) module by saying, "Hi, yahboom" or press the 's' key on your keyboard to begin recording. Then, simply voice your ideas. Wait for the recording to finish—or press the Spacebar to stop recording manually—and the speakers will play an audio prompt announcing that the story is about to begin. The system will then generate the story and convert it into speech for playback. As the narrative shifts between scenes, the background image will automatically update to display visuals relevant to the current setting.

If you wish to end the story, you can press the ESC key. Alternatively, you can interrupt the current story—and restart the recording and story generation process—by saying, "Hello, Xiaoya," to the LLM module.